EXPERIMENT 5

Name-Spandan Deb
Class-D20A
Roll no-20

Aim: Deploying a Voting/Ballot Smart Contract

Theory:

Relevance of require statements in solidity programs

In Solidity, the require statement acts as a guard condition within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt blockchain data.

For example, in a Voting Smart Contract, require can be used to check:

- Whether the person calling the function has the right to vote (require(voters[msg.sender].weight > 0, "Has no right to vote");).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the chairperson before granting voting rights.

Thus, require statements enforce security, correctness, and reliability in smart contracts. They prevent unauthorized access, eliminate invalid operations, and maintain the integrity of the voting process. Additionally, developers can attach error messages to require, which improves debugging and makes contract interaction clearer for users.

Keywords: mapping, storage, and memory

Mapping

A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is:

mapping(keyType => valueType)

For example:

mapping(address => Voter) public voters;

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful in contracts like Ballot, where it is necessary to associate each voter with their data (such as whether they have voted and which proposal they selected).

Unlike arrays:

- Mappings do not have a length property.
- They cannot be iterated over directly.
- They are highly gas-efficient for lookups.

Therefore, mappings are ideal for secure and efficient voter data management.

Storage

In Solidity, storage refers to the permanent memory of the contract stored on the blockchain. Variables declared at the contract level are stored in storage by default.

Key characteristics:

- Data is persistent across transactions.
- It remains available unless explicitly modified.
- Writing to storage consumes gas and is relatively expensive.
- For example, a voter's information saved in the voters mapping remains available throughout the contract’s lifecycle. Storage is essential for maintaining the state of the voting system, such as voter records and proposal vote counts.

Memory

In contrast, memory is temporary storage used only during the execution of a function call. When the function finishes execution, data stored in memory is discarded.

Key characteristics:

- Temporary lifetime.
- Cheaper than storage.
- Used for local variables, function arguments, and intermediate calculations.
- For example, when handling proposal names passed into a function or performing temporary computations, memory is used instead of storage.
- Thus, a smart contract developer must carefully choose between storage and memory to balance cost efficiency and functionality.

Why bytes32 instead of string?

In earlier implementations of the Ballot contract, bytes32 was used for proposal names instead of string. The reason lies mainly in gas efficiency and simplicity.

bytes32 is a fixed-size data type that always stores exactly 32 bytes. This makes:

- Storage is simpler.
- Comparison operations faster.
- Gas consumption is lower.

However, it limits proposal names to 32 characters, reducing flexibility.

On the other hand, string is a dynamically sized data type. It can store text of variable length, making it more user-friendly and readable. However:

- It requires more complex handling inside the Ethereum Virtual Machine (EVM).
- It consumes more gas.
- String comparisons are more expensive.

Modern Ballot contract implementations often use string for better usability, while earlier versions preferred bytes32 for performance optimization.

In summary:

- bytes32 is preferred when performance and gas efficiency are priorities.
- string is preferred when readability and flexibility are more important.

Code-

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity ^0.8.0;

/**

 * @title Ballot

 * @dev Implements voting process along with vote delegation

 */

contract Ballot {


    struct Voter {

        uint weight;

        bool voted;

        address delegate;

        uint vote;

    }


    struct Proposal {
```

```
    string name;
    uint voteCount;
}

address public chairperson;

mapping(address => Voter) public voters;

Proposal[] public proposals;

constructor(string[] memory proposalNames) {
    chairperson = msg.sender;
    voters[chairperson].weight = 1;

    for (uint i = 0; i < proposalNames.length; i++) {
        proposals.push(Proposal({
            name: proposalNames[i],
            voteCount: 0
        }));
    }
}
```

```
function giveRightToVote(address voter) external {
    require(msg.sender == chairperson,
        "Only chairperson can give right to vote.");
    require(!voters[voter].voted,
        "The voter already voted.");
    require(voters[voter].weight == 0,
        "Voter already has right to vote.");

    voters[voter].weight = 1;
}

function delegate(address to) external {
    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0, "No right to vote");
    require(!sender.voted, "Already voted");
    require(to != msg.sender, "Self-delegation not allowed");

    while (voters[to].delegate != address(0)) {
        to = voters[to].delegate;
        require(to != msg.sender, "Delegation loop detected");
    }
```

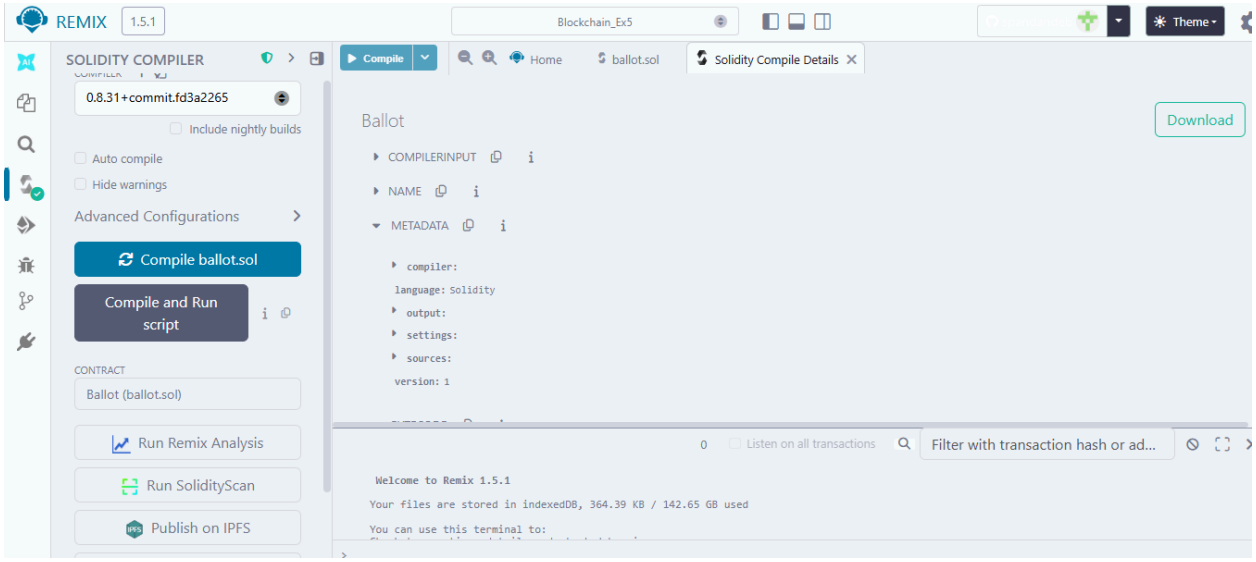
```
    Voter storage delegate_ = voters[to];
    require(delegate_.weight >= 1, "Delegate has no right");

    sender.voted = true;
    sender.delegate = to;

    if (delegate_.voted) {
        proposals[delegate_.vote].voteCount += sender.weight;
    } else {
        delegate_.weight += sender.weight;
    }
}

function vote(uint proposal) external {
    require(proposal < proposals.length,
        "Invalid proposal index");

    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0,
        "No right to vote");
```

```
    require(!sender.voted,
        "Already voted");

    sender.voted = true;
    sender.vote = proposal;

    proposals[proposal].voteCount += sender.weight;
}

function winningProposal() public view returns (uint winningProposal_) {
    uint winningVoteCount = 0;

    for (uint p = 0; p < proposals.length; p++) {
        if (proposals[p].voteCount > winningVoteCount) {
            winningVoteCount = proposals[p].voteCount;
            winningProposal_ = p;
        }
    }
}

function winnerName() external view returns (string memory winnerName_) {
    winnerName_ = proposals[winningProposal()].name;
```

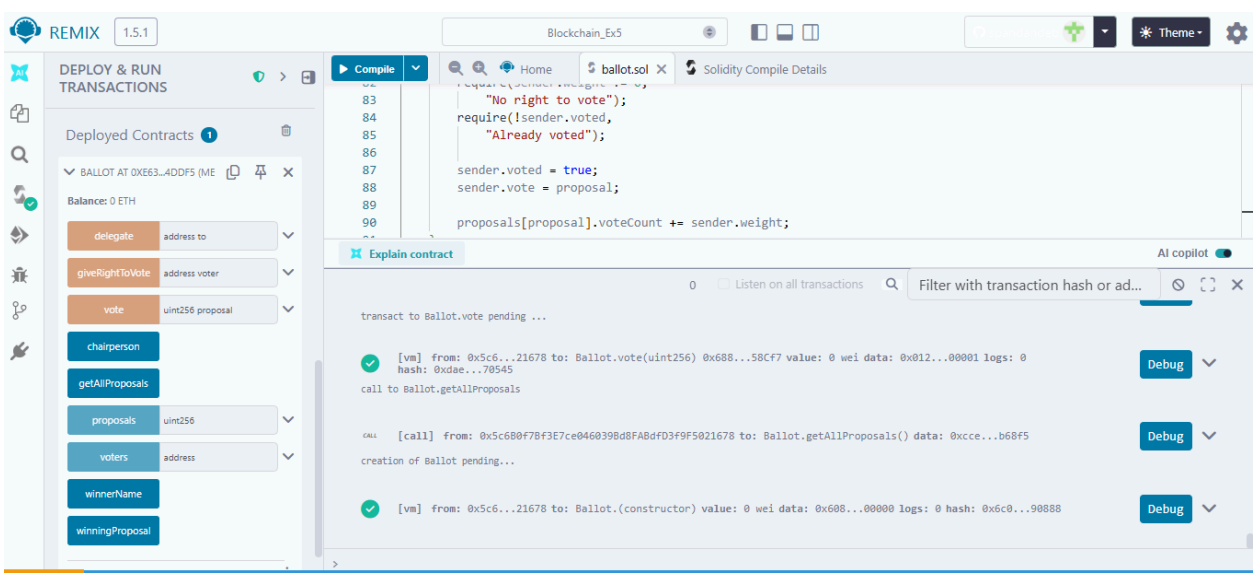
```
    }

    // Spandan Deb D20A
        function getAllProposals() external view returns (string[] memory names, uint[] memory voteCounts) {
        uint length = proposals.length;

        names = new string[](length);
        voteCounts = new uint[](length);

        for (uint i = 0; i < length; i++) {
            names[i] = proposals[i].name;
            voteCounts[i] = proposals[i].voteCount;
        }
    }
}
```

Output-

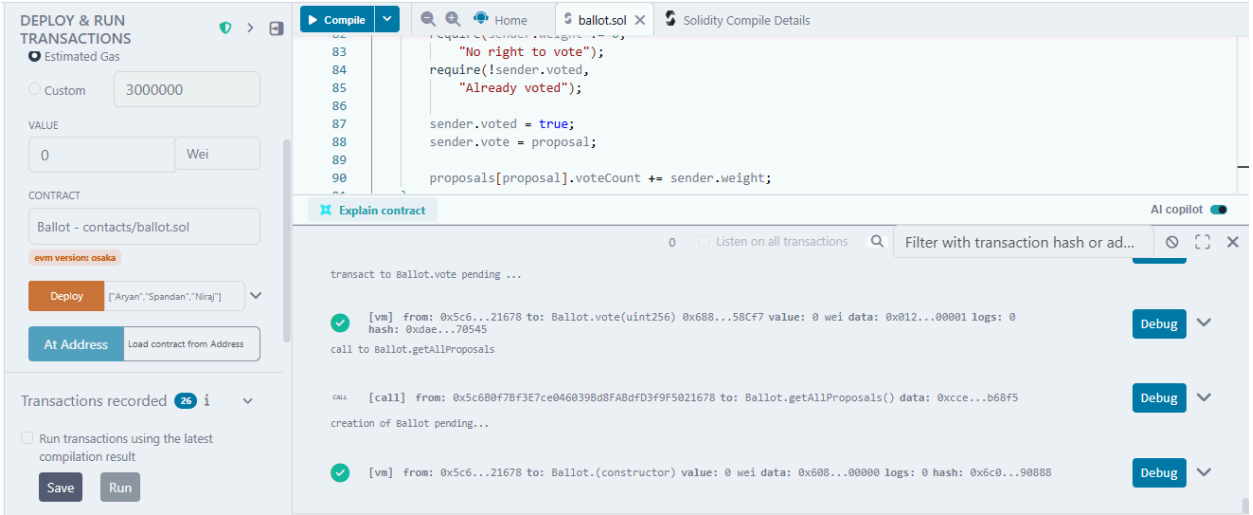
1)Compiled Ballot.sol contract



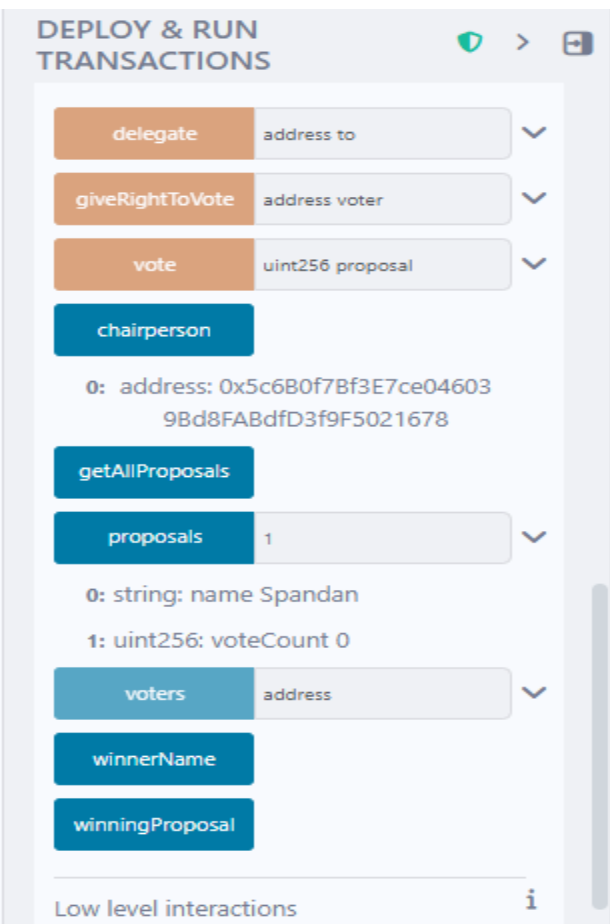
2)Deploying and running of contract



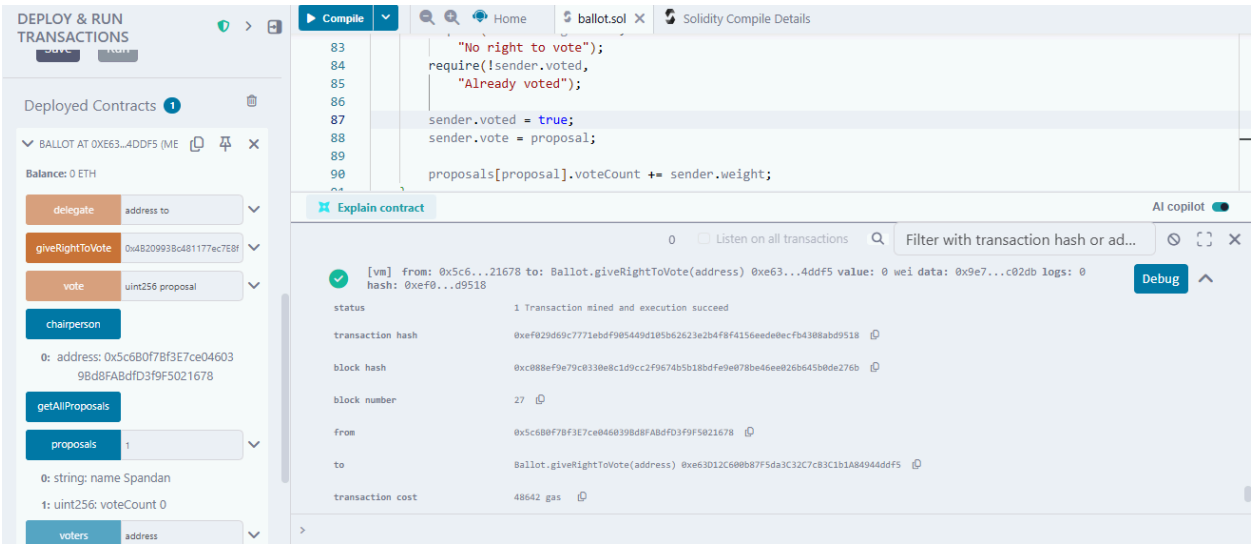
3)Loading the Proposal Candidate's Names (string)



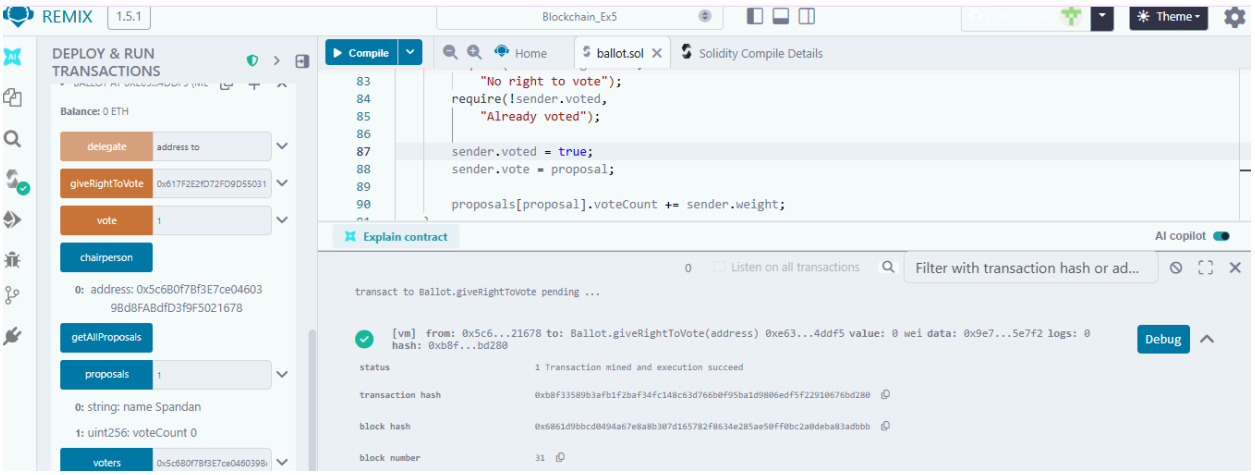
4)Viewing the details of the proposal candidate



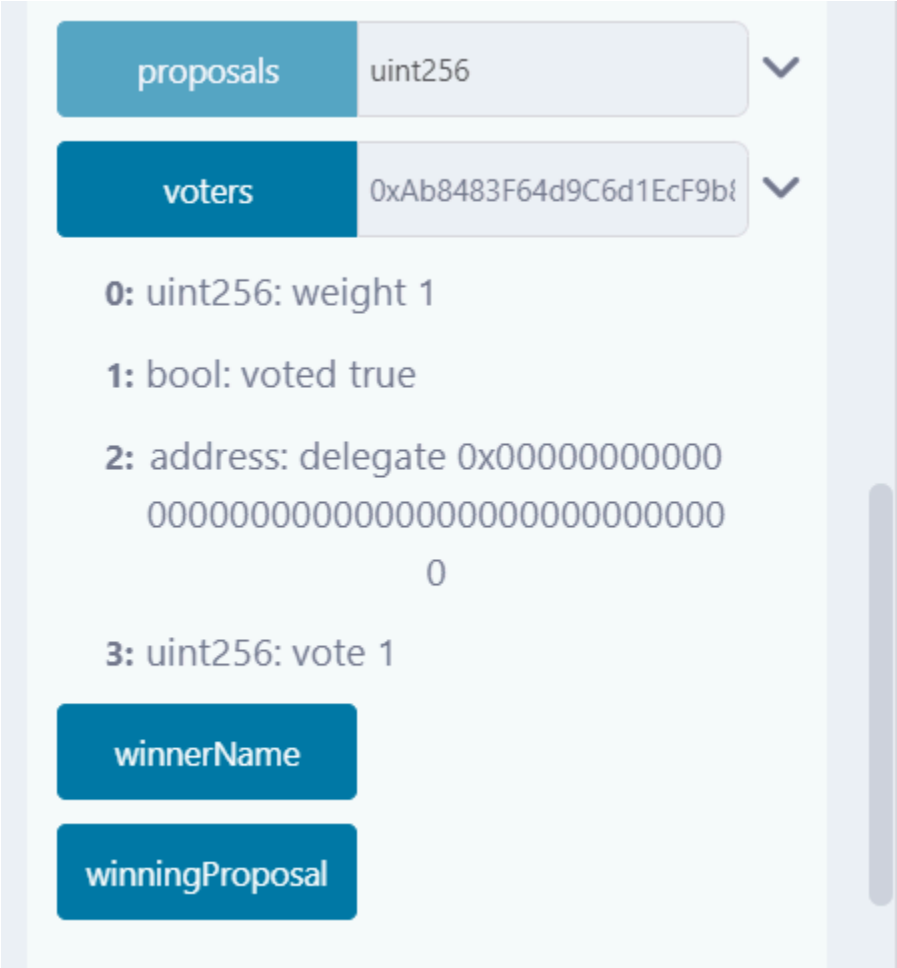
5)Giving right to vote other than the chairperson



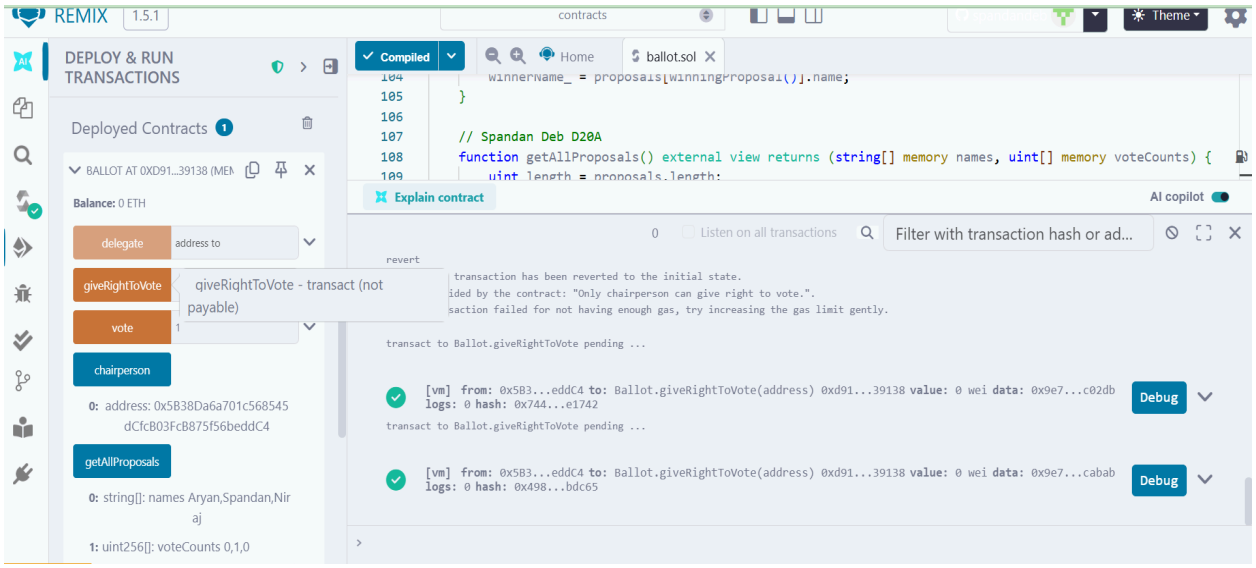
6)Selecting the account other than the chairperson to vote and then writing the proposal candidate's index to vote.

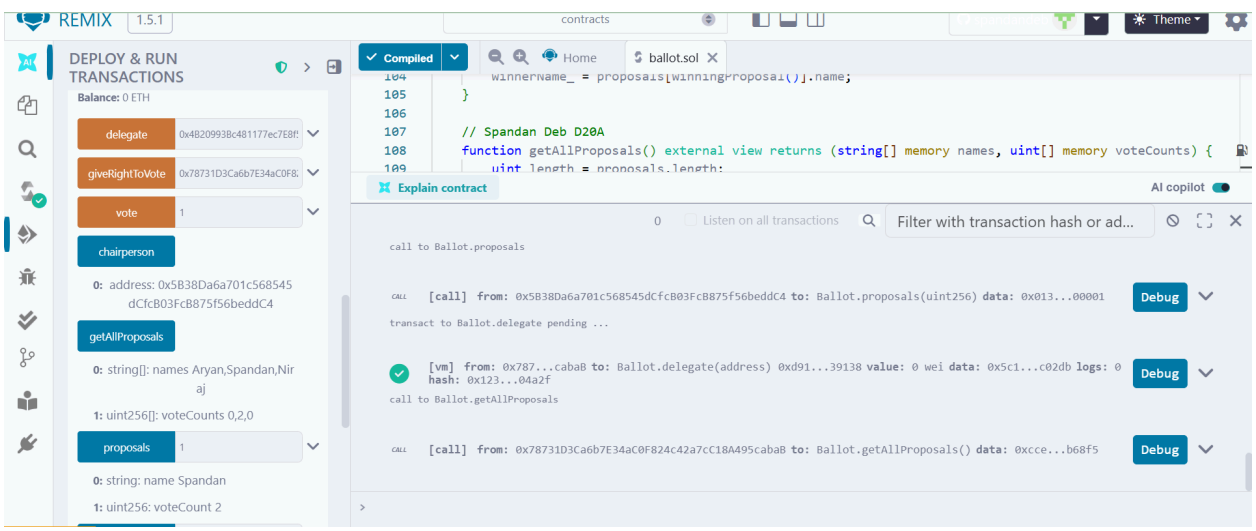


7)We can view the Voter's information



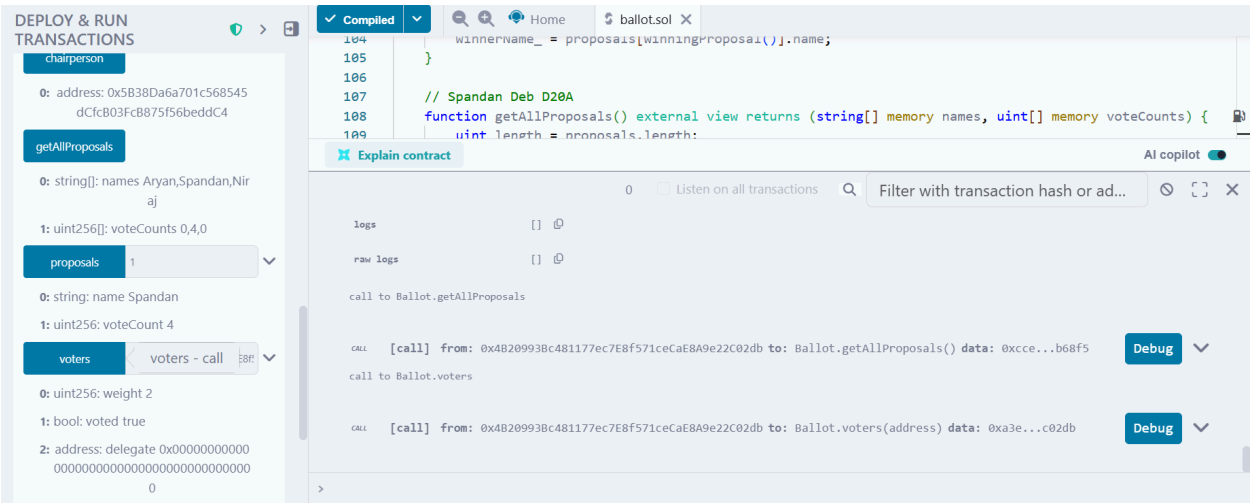
8)Selecting the account of the delegator and then writing the address of the voter to be delegated.

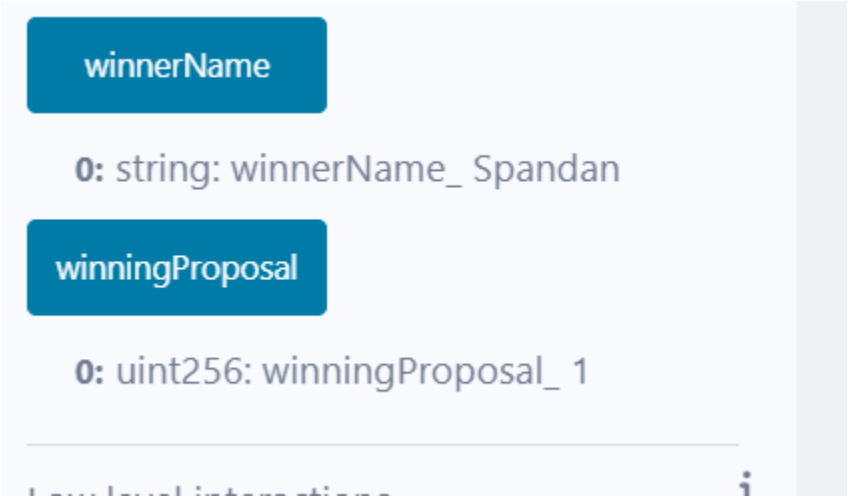




9)Proposal candidates information before the delegates vote

10)In this final screenshot we can see that after the delegation's vote the weight of the vote which was originally 1 (default) is now 2.





Conclusion:-

In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of require statements, mapping, and data location specifiers like storage and memory were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts. The difference between using bytes32 and string for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchain.